WEB APPLICATION / IMPLEMENTATION / ARTIFACTS

StudySprint

Project and Artifact Report

A study planning website with a conversational SprintBot assistant.

Project owner: wrightiv-dotcom **Report date:** 29 October 2026

Project partner: Julian Medina

Live application: https://studysprint-acld.vercel.app/
GitHub repository: https://github.com/wrightiv-dotcom/studysprint
Alternate live site: https://wrightiv-dotcom.github.io/studysprint/

1 Project purpose and user experience

StudySprint helps students turn coursework requirements, deadlines, and available study time into editable milestones and scheduled study sessions. Its purpose is to make a large assignment easier to start and to show whether the estimated work fits before the deadline.

The website contains an overview dashboard, assignment cards, a study schedule, and an availability editor. It uses a night sky background, translucent navy panels, and responsive layouts. Assignment progress and daily study windows are saved in the visitor's browser.

1.1 Typical workflow

1. Set study windows for each day of the week.
2. Select **Chat with SprintBot**. The chat asks for an assignment title, requirements, deadline, and estimated total hours, one message at a time.
3. Review the draft milestones and schedule feasibility displayed in the conversation. Send changes to the requirements to generate a revised draft.
4. Select **Review milestones & schedule**, edit the draft as needed, and approve it before sessions are added to the study schedule.
5. Mark sessions complete and replan unfinished work through the assignment review flow.

1.2 Implemented planning scope

Local planning uses assignment templates and task lists; it is not a language model. If the requirements contain two to eight suitable task lines, those lines become the milestones. Otherwise, writing, coding, exam, presentation, or general templates supply starter steps. Estimates are distributed in whole minutes. The chat validates dates and total estimates between 0.5 and 200 hours.

The live Vercel and GitHub Pages editions currently provide local planning. The repository also includes an optional Express backend and a tool-calling AI agent. Live AI requires a separately running backend with a privately configured provider key. The chat clearly indicates when AI is unavailable.

2 Architecture and project artifacts

2.1 Application structure

The static frontend loads HTML, CSS, and JavaScript from `dist/`. Application state is stored under `studysprint-v1` in `localStorage`. The planner generates draft milestones; the shared scheduler checks them against availability and other approved sessions. Approval remains a user action.

Artifact	Purpose
`dist/index.html`	Dashboard structure, assignment dialogs, navigation, and report links.
`dist/style.css`	Responsive layout and night sky theme.
`dist/script.js`	Browser state, assignments, progress, approval, and replanning.
`dist/scheduler.js`	Shared availability, conflict, deadline, and shortfall calculations.
`dist/sprint-planner.js`	Local templates, task extraction, input validation, and effort allocation.
`dist/sprintbot.js`	Conversational questions, message history, revisions, and draft review handoff.
`dist/sprintbot.css`	Chat launcher, dialog, messages, and message composer styling.
`dist/assets/clouds-night-sky.gif`	Animated visual background.
`docs/`	Published copy used by GitHub Pages.
`vercel.json`	Selects the static `dist/` output to prevent a Vercel root-page 404.
`studysprint.html`	Downloadable HTML edition in the repository root; styles and scripts are embedded.
`scripts/build-html-download.cjs`	Rebuilds the root HTML edition. The background image and report links use internet resources.
`scripts/prepare-pages.cjs`	Rebuilds the HTML edition and copies `dist/` into `docs/`.
`server/app.cjs`, `server/agent.cjs`	Optional Express API and bounded model tool-call loop.
`tests/`	Planner, agent, API, and chat-flow checks.
`report/studysprint-report.tex`	Editable LaTeX source of this report.
`output/pdf/studysprint-report.pdf`	PDF compiled from the LaTeX source. Copies are served from `dist/report/` and `docs/report/`.

2.2 Data boundary

Saved assignments are local to a browser and origin; Vercel and GitHub Pages have separate storage. Clearing browser data can remove assignments. There is no account system or cross-device backup. Credentials and environment files are excluded from public source history.

3 Scheduling and optional AI backend

3.1 Schedule validation

The scheduler searches available daily windows up to the deadline, subject to a 730-day search limit. On the current day, it starts no earlier than the next applicable quarter-hour. Other approved, unfinished sessions occupy time. Sessions last up to 60 minutes and can be shorter at a window boundary or when finishing a milestone.

If work cannot fit, the scheduler reports a shortfall in minutes. Invalid estimates and infeasible plans cannot be approved unchanged. The assignment review and replanning flow accounts for completed work. Generating a chat draft does not itself approve the proposal or replace another approved assignment.

3.2 Example

A four-hour research paper may produce research, outline, draft, and revision milestones of one hour each. Two free two-hour windows before the deadline can hold four one-hour sessions, assuming no conflicts. If only one two-hour window is available, the plan is short by two hours. The student must adjust the plan or available time before approval. This example illustrates the algorithm, not a claim that every paper takes four hours.

3.3 Optional Express and model integration

The optional Express application serves the frontend and exposes `/api/health`, `/api/config`, and `/api/plan`. Its three tools are `read_assignment`, `get_availability`, and `check_schedule`. It must read the assignment and availability before validating a proposal. Tool feedback allows revision; the agent cannot approve plans, execute arbitrary code, or save work by itself.

The loop permits at most six model turns, eight tool calls, and three schedule checks. The model must keep the student's effort budget and return two to eight milestones. The independent scheduler validates the proposal rather than trusting a model's feasibility claim.

3.4 Credentials, consent, and limitations

The server reads `OPENAI_API_KEY` from a private environment variable. Keys must not appear in browser scripts, GitHub, or this report. Before AI planning, the chat asks permission to send the selected assignment, availability windows, occupied times, and time zone to the server and OpenAI. The provider request uses `store: false`; that setting is not a promise of zero provider retention.

The Express API includes request validation, origin checks, rate limits, a concurrency limit, and a 90-second timeout. In-memory rate limits reset when the process restarts. The backend can be run locally or deployed using `render.yaml`; `DEPLOYMENT.md` explains activation. **No real-provider request or live Express deployment is claimed as verified in this report.** Static Vercel hosting does not activate that backend.

4 Validation, deployment, and reproduction

4.1 Verification evidence

The project has 18 automated checks: six planner tests, six agent tests, four API tests, and two chat-flow tests. The suite covers templates, explicit tasks, minute allocation, invalid inputs, schedule conflicts, infeasible workloads, bounded tool use, API errors, and key isolation. Chat tests cover conversational collection, invalid replies, draft revisions, review before approval, and declining AI consent. Agent tests use simulated provider responses and spend no API credits.

The suite passed using `node -test -test-isolation=none tests/*.test.cjs`. Disabling process isolation accommodates the local execution environment. The normal repository command is `npm test`. These checks do not constitute a full security or accessibility audit, and they do not verify a real model account or billing setup.

4.2 Deployment and linked deliverables

The Vercel deployment at https://studysprint-acld.vercel.app/ was checked successfully after adding `vercel.json`. Its configuration explicitly selects `dist/` as the published output, fixing the previous root-page 404. GitHub Pages publishes `docs/` from `main` at https://wrightiv-dotcom.github.io/studysprint/. The footer on both sites links to the compiled report and LaTeX source.

The root `studysprint.html` is provided as a repository artifact. It is not advertised through a website download button. Its CSS and JavaScript are included in the file; the animated background and linked reports still require internet access.

4.3 Reproduction instructions

1. Clone https://github.com/wrightiv-dotcom/studysprint. Run `npm ci`, then `npm start` for the optional Express application. Static local planning can also be opened from `dist/index.html`.
2. Run `npm test`. Edit website files in `dist/`, then run `npm run prepare:pages` to update `docs/` and the root HTML edition.
3. Compile this report with Tectonic:
   `tectonic -o output/pdf report/studysprint-report.tex`
4. Copy the PDF and LaTeX source into `dist/report/`, run `npm run prepare:pages`, and commit the source and deliverables to `main`.
5. Confirm the relevant Vercel and Pages deployments succeed and that the live report links open the updated PDF.

4.4 Project links

- **Live app:** https://studysprint-acld.vercel.app/
- **GitHub source and artifacts:** https://github.com/wrightiv-dotcom/studysprint
- **Alternate live site:** https://wrightiv-dotcom.github.io/studysprint/
- **Deployment configuration reference:** Vercel vercel.json documentation.